Claude Code

Workshop prático · NEI



Vicente Barros & Sebastião Teixeira

Universidade de Aveiro · 12 de novembro de 2026

O plano de hoje

00

Fundamentos

ex00

Agentes, contexto, permissões, Claude Code vs OpenCode

01

Trabalhar bem

ex01–04

Comandos, plan mode, prompts, CLAUDE.md, memória

02

Estender

ex05–08

Hooks, subagentes, skills, MCP

03

Escalar

ex09–11

Modelos, paralelismo, CI, personalização, plugins

Teoria curta → exercício logo a seguir. Fio condutor: o contexto é o recurso escasso.

00

Fundamentos

Antes do exercício 00

A IA não vai fazer o curso por vocês

−17

pontos percentuais

no teste de compreensão para quem aprendeu uma biblioteca nova com ajuda de IA. E sem ganho de tempo significativo.

Quem aprendeu bem com IA

- Fazia perguntas de seguimento
- Pedia explicações, não só código
- Usava a IA para perceber, não só para produzir

Usem-na como tutor, não como ghostwriter. No exame estão sozinhos.

Fonte: Shen & Tamkin, Anthropic (jan. 2026). Ensaio aleatorizado com 52 programadores a aprender a biblioteca Trio.

Como chegámos aqui



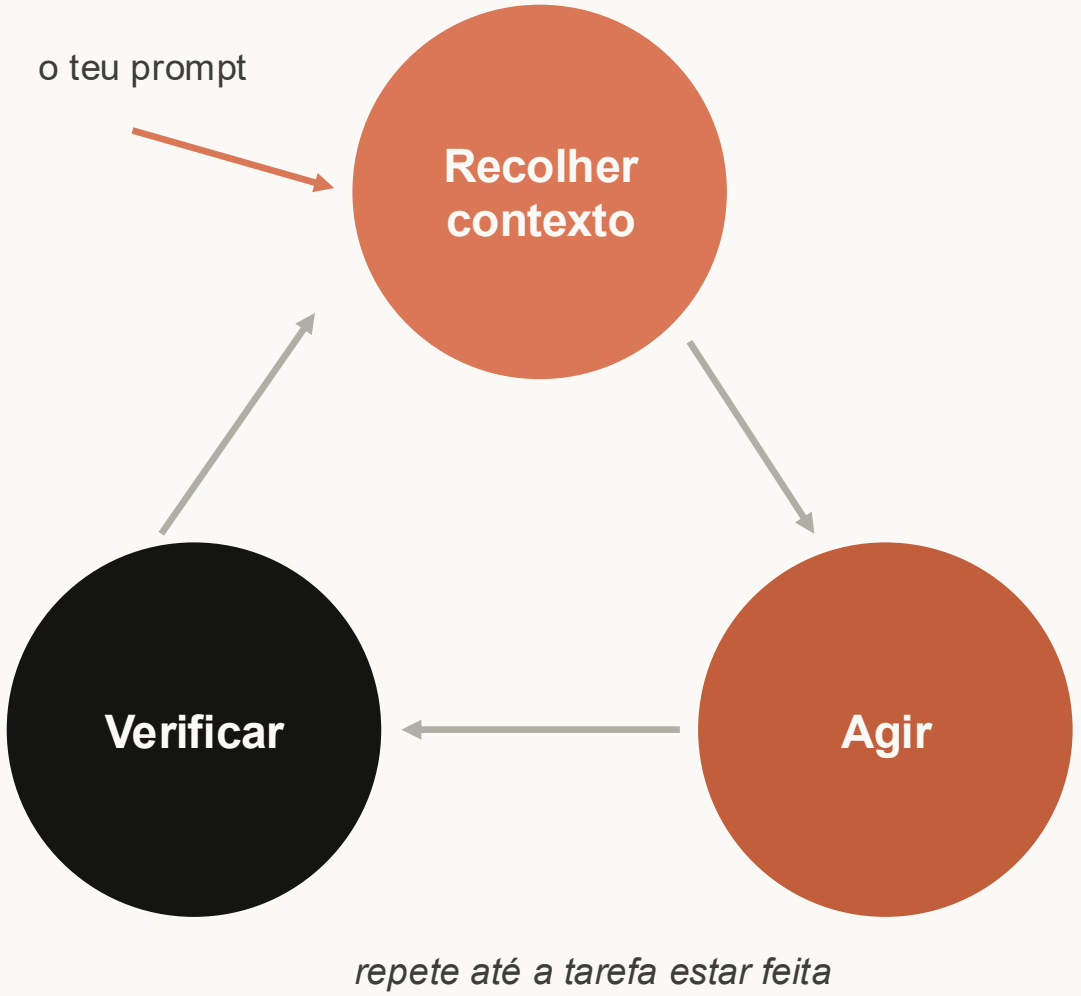
Sugere uma linha → Responde a perguntas → **Age sozinho**

O que é um agente?

«Um LLM a usar ferramentas, em ciclo.»

1. **Recolhe contexto**
   lê ficheiros, pesquisa o código
2. **Age**
   edita ficheiros, corre comandos
3. **Verifica**
   testes, output, erros

Tu também estás no ciclo: Esc interrompe, uma mensagem corrige o rumo.



Anatomia de um agente



Modelo

Raciocina e decide o próximo passo.

Haiku · Sonnet · Opus · Fable

+



Ferramentas

Ler e editar ficheiros, pesquisar, shell, web.

Sem ferramentas, só escreve texto.

+



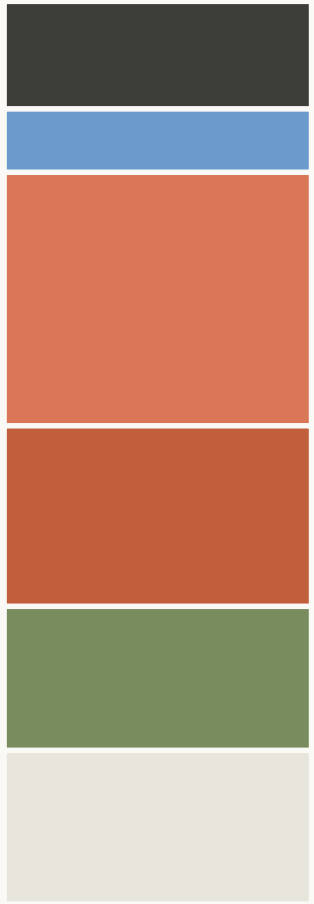
Harness

O programa à volta do modelo: corre as ferramentas, gere o contexto e as permissões.

Claude Code e OpenCode são harnesses.

O mesmo modelo num harness diferente comporta-se de forma diferente.

A janela de contexto



Prompt de sistema e ferramentas

CLAUDE.md e memória

Ficheiros lidos

Output de comandos

Conversa

Livre

Ilustrativo. Vê o teu com /context.

É a memória de trabalho
Tudo o que o modelo vê neste momento: instruções, ficheiros, output, conversa.

É finita, e a qualidade cai
Quanto mais cheia, pior o desempenho. Ruído também conta.

Compactação automática
Perto do limite, a conversa é resumida. Perde-se detalhe.

/clear entre tarefas
Uma tarefa nova não deve pagar o histórico da anterior.

Permissões: quem decide o quê

Manual

Pergunta antes de editar ficheiros ou correr comandos.

Aceitar edições

Edita sem perguntar. Comandos ainda pedem autorização.

Plano

Só lê e propõe um plano. Não mexe no código.

Auto

Um classificador bloqueia ações arriscadas. É o modo padrão.

`Shift+Tab`	alterna entre os modos
`Esc Esc`	checkpoints: desfaz edições de ficheiros
`settings.json`	regras allow / deny por comando ou ficheiro

`bypassPermissions`
Só num contentor descartável. Checkpoints não desfazem deploys nem bases de dados.

Claude Code vs OpenCode

	Claude Code	OpenCode
Origem	Anthropic	Open source
Modelos	Claude: Haiku, Sonnet, Opus, Fable	Qualquer fornecedor, incluindo modelos locais
Planear	Plan mode (Shift+Tab)	Agentes build / plan (Tab)
Instruções	CLAUDE.md (lê AGENTS.md se não houver)	AGENTS.md
Extensões	Skills, hooks, subagentes, MCP, plugins	Agentes, comandos, skills, MCP, plugins
Onde corre	Terminal, VS Code, desktop, web, CI	Terminal, desktop, web



Os conceitos transferem-se: loop, ferramentas, contexto, permissões. A ferramenta é um detalhe.

Mãos à obra: setup

→ ex00

```
# clonar e preparar
git clone https://github.com/sebastiaoteixeira/workshop-claude
cd workshop-claude
make test

# abrir o Claude Code com o plugin do workshop
claude --plugin-dir ./plugin
> /ex 00
```

O plugin do workshop

`/ex NN`
Põe o repositório no início do exercício e corre os testes

`/hint NN k`
Mostra a dica k do exercício

`/debrief NN`
A pergunta final do exercício

01

Trabalhar bem

Antes dos exercícios 01–04

Comandos essenciais

Slash commands

`/init`	gera um CLAUDE.md inicial
`/context`	o que está a ocupar a janela
`/clear`	começa de novo, sem histórico
`/compact <foco>`	resume a conversa
`/model · /effort`	modelo e nível de esforço
`/rewind`	volta atrás no código e/ou conversa
`/resume`	retoma uma sessão anterior
`/mcp`	estado dos servidores MCP

Atalhos

`Esc`	para o Claude a meio
`Esc Esc`	abre o menu de rewind
`Shift+Tab`	muda o modo de permissões
`@ficheiro`	inclui um ficheiro no prompt
`! comando`	corre um comando na shell
`colar imagem`	screenshots de erros ou designs

Explorar → Planear → Implementar → Commit

→ ex02



Explorar

Lê o código relevante. Ainda sem editar.



Planear

Plan mode: propõe a abordagem. Tu revês.





Implementar

Executa o plano e verifica com testes.





Commit

Mensagem clara, PR se for o caso.

Usa plan mode quando…

- o código não é teu
- a alteração toca vários ficheiros
- não sabes bem qual é a abordagem

Salta-o quando…

- a correção é óbvia e de uma linha
- consegues descrever o diff numa frase

Prompts: dá-lhe o que ele não vê

✗ **Vago**

«corrige o bug do checkout»

✓ **Específico**

«O checkout falha para utilizadores com cartões expirados. O código está em src/payments/. Investiga e corrige; o teste test_expired_card tem de passar.»



@ficheiros
aponta para o código certo



Imagens
screenshot do erro ou do design



Logs
cat erro.log | claude -p "…"



Decisões
o que foi combinado fora do código

Delega, não dites: dá contexto e objetivo, deixa os detalhes com ele.

Dá-lhe uma forma de verificar

→ ex04

«Feito» só conta se alguma coisa verificou.

Sem uma verificação, o Claude decide sozinho quando acabou. Com uma, tem um alvo.

```
> corre os testes no fim e
>   continua até passarem
```

Testes
o melhor alvo: escreve-os primeiro (TDD)

Linter e typecheck
erros apanhados logo após cada edição

Exit code do build
passa ou não passa, sem interpretação

Screenshot vs mockup
para trabalho de interface

Outro revisor
um subagente que não escreveu o código

Corrigir o rumo

Esc

Para a meio. Mantém o trabalho feito até ali, e podes redirecionar.

Esc Esc · /rewind

Volta a um prompt anterior: código, conversa ou ambos. Experimentar fica barato.

/clear

Corrigiste duas vezes e continua errado? Recomeça com um prompt que já inclui o que aprendeste.

A terceira correção é só mais ruído.

O contexto já tem duas tentativas falhadas, e o Claude continua a relê-las.

Memória: o CLAUDE.md

→ ex03

Organização
política gerida pela empresa

`~/.claude/CLAUDE.md`
tu, em todos os projetos

`./CLAUDE.md`
a equipa, versionado no git

`./CLAUDE.local.md`
tu, só neste projeto

Todos se somam no contexto, do geral para o específico.

O que lá pôr

- Comandos exatos de teste, lint e build
- Onde o código engana: padrões antigos a não copiar
- Decisões que não estão no código
- Tudo o que já corrigiste duas vezes

< 200 linhas. /init gera um ponto de partida; depois o trabalho é cortar. Regras só para algumas pastas vão para **`.claude/rules/`**.

Memória automática e sessões

CLAUDE.md

Escreves tu

Regras e convenções. Carregado sempre, inteiro.

Auto memory

Escreve o Claude

Aprendizagens e preferências tuas. MEMORY.md, primeiras 200 linhas.

1

Cada sessão começa do zero
Só o CLAUDE.md e a memória atravessam sessões.

2

A compactação perde detalhe
Regras que têm de durar vão para o CLAUDE.md, não para a conversa.

3

Podes voltar atrás
claude --continue e /resume retomam uma sessão anterior.

02

Estender o Claude Code

Antes dos exercícios 05–08

O mapa das extensões

Plugin empacota tudo isto numa instalação

CLAUDE.md

Sempre presente

Custo: em cada pedido

Skills

Workflows e conhecimento

Custo: baixo até ser usada

MCP

Ligação a sistemas externos

Custo: baixo até ser usado

Subagentes

Trabalho em contexto isolado

Custo: isolado

Hooks

Corre sozinho em eventos

Custo: zero

Output styles

Tom e formato das respostas

Custo: em cada pedido

Hooks: pedido vs garantia

→ ex05

No CLAUDE.md

«Nunca edites o .env»

É um pedido. O modelo pode esquecer-se.

Num hook PreToolUse

Bloqueia a edição antes de acontecer

É uma garantia. O harness executa-o sempre.

Eventos

`SessionStart`	`UserPromptSubmit`
`PreToolUse`	`PostToolUse`
`Notification`	`Stop`

```
# .claude/settings.json
"hooks": { "Notification": [{
  "hooks": [{ "type": "command",
    "command": "say 'Preciso de ti'" }]
}]}
```

Exemplo: avisa-te quando o Claude está à espera.

Subagentes

→ ex06

Conversa principal

O teu contexto fica limpo: só recebe o resumo.

tarefa →

← **resumo de 10 linhas**

Subagente

Contexto próprio. Lê 40 ficheiros, corre pesquisas, testa hipóteses.

Pesquisas amplas
o output fica fora do teu contexto

Revisor independente
não viu como o código foi escrito

Ferramentas limitadas
tools: Read, Grep, Glob

Modelo mais barato
model: haiku para tarefas simples

Skills

→ ex07

```
# .claude/skills/explicar/SKILL.md
---
name: explicar
description: Explica código passo a
  passo. Usa quando o utilizador quer
  perceber um ficheiro ou função.
---
1. Lê o ficheiro e as dependências
2. Explica o fluxo com um diagrama
3. Acaba com 3 perguntas de revisão
```

1 **Só a descrição fica no contexto**
O conteúdo inteiro só carrega quando é usada.

2 **Duas formas de a invocar**
Tu, com /explicar, ou o Claude sozinho, quando o pedido bate com a descrição.

3 **A descrição é o que importa**
Vaga ou parecida com outra: a skill não dispara.

4 **Referência ou ação**
Um guia de estilo vs um /deploy. Com efeitos secundários: disable-model-invocation.

MCP: ligar o Claude ao mundo

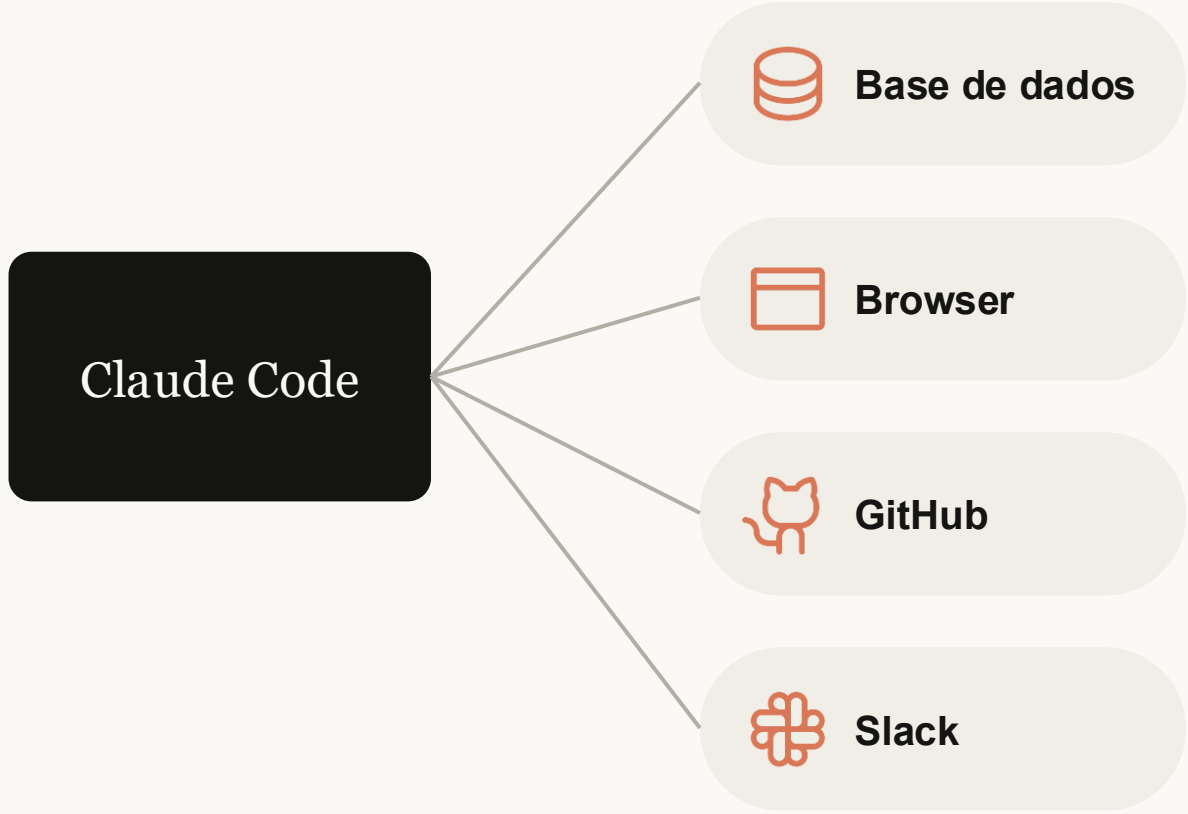
→ ex08

Um protocolo aberto para ligar ferramentas e dados externos.

- As definições das ferramentas só carregam quando são precisas
- claude mcp add para ligar, /mcp para ver o estado

O MCP dá acesso. Uma skill ensina a usá-lo bem.

Um servidor MCP é código a correr na tua máquina. Só instala os que conheces.



Qual usar quando

Sinal	Usa
O Claude errou a mesma convenção duas vezes	**CLAUDE.md**
Escreves o mesmo prompt pela terceira vez	**Skill**
Copias dados de um separador que o Claude não vê	**MCP**
Uma tarefa lateral enche a conversa de output	**Subagente**
Tem de acontecer sempre, sem pedir	**Hook**
Um segundo repositório precisa do mesmo setup	**Plugin**

Não configures tudo no primeiro dia. Cada funcionalidade tem um sinal que diz quando a adicionar.

03

Escalar

Antes dos exercícios 09–11

O modelo certo para cada tarefa

Haiku

Rápido e barato

Tarefas simples, subagentes de pesquisa

custo ●○○○

Sonnet

O dia a dia

A maior parte do código

custo ●●○○

Opus

Raciocínio complexo

Arquitetura, bugs difíceis. O padrão.

custo ●●●●

Fable

O mais capaz

As tarefas mais longas e difíceis

custo ●●●●

Esforço /effort

`low` `medium` `high` `xhigh` `max`

padrão: medium

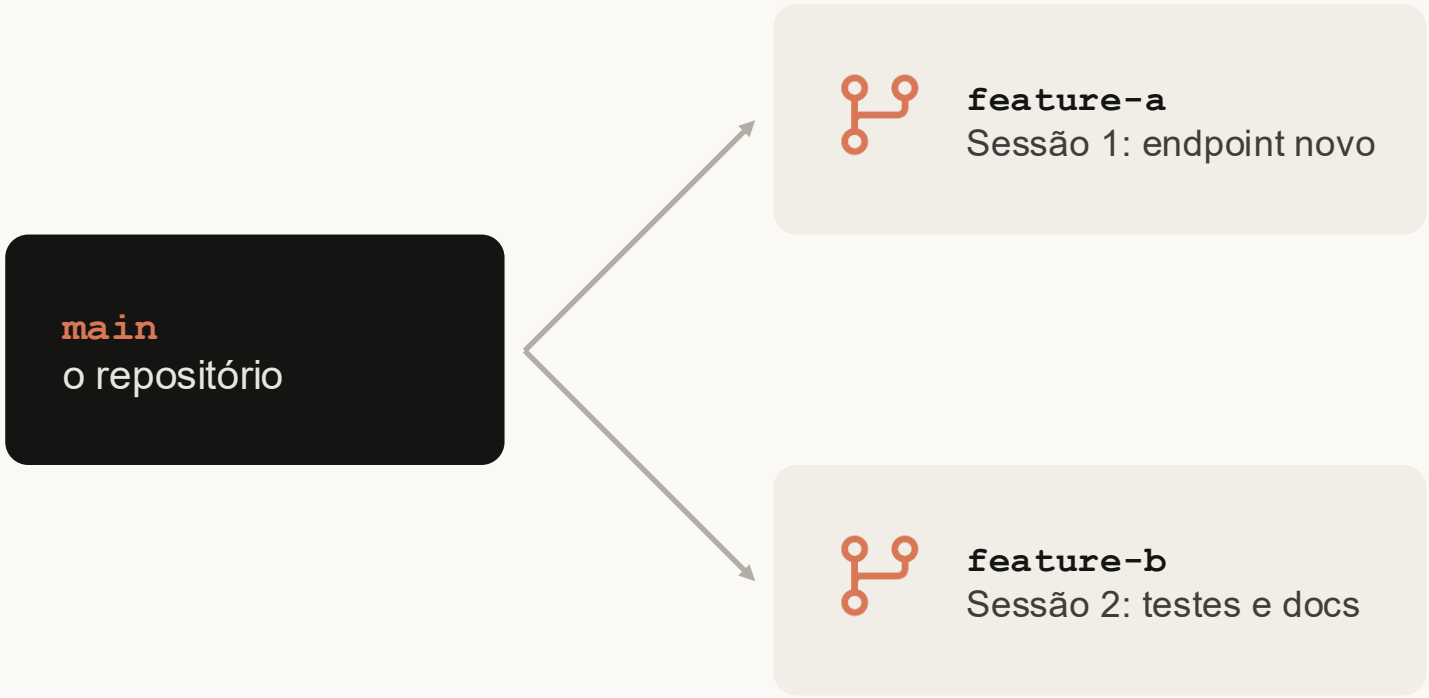
`ultrathink` num prompt: mais raciocínio só nesse turno

`opusplan` Opus planeia, Sonnet executa

`/usage` quanto estás a gastar

Trabalhar em paralelo

→ ex09




```
# uma pasta por branch
git worktree add ../a -b feature-a
git worktree add ../b -b feature-b

# um Claude em cada
cd ../a && claude
```

- Cada sessão tem o seu contexto e os seus ficheiros
- Divide por fronteiras claras para evitar conflitos no merge
- Subagentes em background para trabalho mais pequeno

Headless e CI

→ ex10

```
# modo não interativo: um prompt, uma resposta
cat erro.log | claude -p "qual é a causa provável?"

# para scripts e pipelines
claude -p "…" \
  --output-format json \
  --allowedTools "Read,Grep" \
  --permission-mode dontAsk
```

`-p`
corre sem interface e termina

`--output-format json`
resultado fácil de interpretar

`--allowedTools`
só as ferramentas que a tarefa precisa

`GitHub Actions`
o mesmo, a correr em cada PR

Personalização

Quem ganha, de cima para baixo

Gerido pela organização	não se sobrepõe
Flags na linha de comando	só esta sessão
`.claude/settings.local.json`	tu, neste projeto
`.claude/settings.json`	a equipa, no git
`~/.claude/settings.json`	tu, em todo o lado

```
{
  "model": "sonnet",
  "permissions": {
    "allow": ["Bash(make test)"],
    "deny": ["Bash(git push:*)"]
  }
}
```

Output styles
tom e formato

Status line
info no rodapé

Keybindings
os teus atalhos

Plugins: empacotar e partilhar

→ ex11

```
plugin/
  .claude-plugin/plugin.json
  skills/       # /plugin:skill
  agents/       # subagentes
  hooks/        # hooks
  .mcp.json     # servidores MCP
```

Tudo o que configuraste numa só instalação.

- Comandos com namespace: /nei-workshop:hint
- Marketplaces para distribuir pela equipa
- O plugin deste workshop é um exemplo: /ex, /hint, /debrief

```
# instalar a partir de um marketplace
claude plugin marketplace add ./
claude plugin install nei-workshop@workshop-claude-nei
```

5 regras para levar

1. O contexto é escasso
   /clear sem medo
2. Dá-lhe uma forma de verificar
   testes, lint, build
3. O que é obrigatório vai para um hook
   não para o prompt
4. O que se repete vai para o CLAUDE.md
   ou para uma skill
5. Percebe o que ele escreve
   o exame é teu

Obrigado! Perguntas?



Documentação do Claude Code
code.claude.com/docs



OpenCode
opencode.ai/docs



IA e formação de competências (estudo)
anthropic.com/research/AI-assistance-coding-skills



Context engineering para agentes
anthropic.com/engineering/effective-context-engineering-for-ai-agents



Este workshop
github.com/sebastiaoteixeira/workshop-claude